

claude-windows / 6e9d0958-a2ec-4a8e-bb2d-6cce9833c527

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\6e9d0958-a2ec-4a8e-bb2d-6cce9833c527\subagents\agent-aebd6855b594a6030.jsonl`  
- SHA-256: `0e81dd1b1fb34ced450b8b6b4d1daa71a8a8d4e8e4c4597bd15ed0c747e56f4c`  
- Source modified: `2026-07-04T00:48:54+00:00`  
- Imported at: `2026-07-05T16:48:26+00:00`  
- project: `subagents`  
- session_id: `6e9d0958-a2ec-4a8e-bb2d-6cce9833c527`
```

Transcript

1. user (2026-07-04T00:48:43.182Z)

Code-review VERIFIER. Repo: C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer. PR #458 branch = origin/codex/a7-a9-quality-chain. The PR-branch fusion_golden.py is at C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/6e9d0958-a2ec-4a8e-bb2d-6cce9833c527/scratchpad/pr458_fusion_golden.py ? Read it.

CANDIDATE: the new `--missing-fusion-only` flag (param `missing\_fusion\_only` in run()) is a functional no-op ? it changes only a log message, not which videos get re-extracted.

In run()'s per-spec loop, read the exact branch sequence (around the `existing`/`existing\_full`/`missing\_fusion\_only` checks). The claim is:

```
- `if missing_fusion_only and not fresh and existing_full:` ? skip (append+continue)  
- `if not fresh and existing and existing_full:` ? skip (append+continue)  
- `if not fresh and existing and not existing_full:` ? print "re-extracting" (fall through to extract)
```

VERIFY: enumerate the behavior for a video across all combinations of (missing_fusion_only ? {T,F}) × (existing ? {T,F}) × (existing_full ? {T,F}), with fresh=False. For each, does the video get SKIPPED or RE-EXTRACTED? Then answer: does setting missing_fusion_only=True EVER change whether a video is skipped-vs-extracted compared to missing_fusion_only=False? Or does it only change the log line printed? If the flag never changes extraction behavior, it's a no-op (dead/ misleading CLI surface). Note the flag's --help text: "skip videos whose existing JSON already has full shuttle+person fusion schema" ? is that ALSO exactly what the default (no-flag) path does?

Return one verdict: CONFIRMED (flag is a no-op ? the default already does the same skip/extract; only the log message differs ? show the two branches), PLAUSIBLE, or REFUTED (name the input where the flag changes extraction behavior). Final message: verdict, 3-5 sentences with the branch logic, one-line fix.

2. assistant (2026-07-04T00:48:44.949Z)

I'll analyze the fusion_golden.py file to verify whether the `--missing-fusion-only` flag is truly a functional no-op.

3. user (2026-07-04T00:48:45.228Z)

```
1      """Offline golden-set person-feature extractor ? the fusion-P3 measurement's GPU step.  
2  
3      Mirrors `distill_local`'s CSV-driven flow but ADDS the ~5 person-cue features per  
shuttle  
4      candidate window, by running `PersonDetector` over the ORIGINAL video. It reuses the  
5      EXISTING golden trajectory CSVs (so **no WASB re-run**), needs **no Gemini and no DB**,  
and  
6      writes one small JSON per video. The LOVO comparison (`fusion_compare.py`) then runs  
100% on
```

```

7      CPU offline. So the GPU touches exactly one thing ? person detection ? and the lift is
8      measured by the shipped experiment harness; person features enter only as feature
columns the
9      learned scorer weights (no special-casing, default-OFF until the golden ?F1 justifies
them).
10
11     Run (on the GPU box):
12     python -m backend.eval.fusion_golden --manifest output/human_lovo_manifest.json
--out-dir output
13
14     Each output `output/<name>_golden_features.json` is the replayable feature substrate ?
re-run
15     `fusion_compare` with different variants/thresholds forever without re-detecting on the
GPU.
16     """
17
18     from __future__ import annotations
19
20     import argparse
21     import json
22     import os
23     import time
24     from typing import Any, Dict, List, Optional, Sequence
25
26     from backend.eval import distill_local
27     from backend.eval.calibrate_wasb import load_golden_gts
28     from backend.eval.golden_manifest import load_resolved_manifest
29     from backend.eval.training import label_candidates
30     from backend.pipeline.detectors import fusion_features as ff
31     from backend.pipeline.detectors import rally_gate
32     from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
33     from backend.utils.run_telemetry import RunTelemetry
34
35     # The 5 fps/resolution-invariant shuttle features distill_local already computes.
36     SHUTTLE_FEATURE_NAMES = list(distill_local.FEATURE_NAMES)
37     # The 5 person-cue features (fusion_features.PERSON_FEATURE_NAMES).
38     PERSON_FEATURE_NAMES = list(ff.PERSON_FEATURE_NAMES)
39     SOURCE_MANIFEST_ENV = "RALLY_GOLDEN_SOURCE_MANIFEST"
40
41
42     def is_full_fusion_feature_file(path: str) -> bool:
43         """True when an existing golden feature JSON already has shuttle+person cue dicts.
44
45         Corpus-growth runs can leave leaner windowing-only JSONs with the same filename
pattern.
46         Those are useful for gts/windowing analysis, but they are not the replayable fusion
47         substrate that ``fusion_compare`` and ``ablation`` need.
48         """
49
50         try:
51             with open(path, encoding="utf-8") as f:
52                 data = json.load(f)
53         except (OSError, json.JSONDecodeError, TypeError):
54             return False
55         candidates = data.get("candidates")
56         if not isinstance(candidates, list):
57             return False
58         if not candidates:
59             return True
60         return all(
61             isinstance(candidate, dict)
62             and "shuttle" in candidate
63             and "person" in candidate
64             for candidate in candidates
65         )

```

```

66
67
68 def load_source_video_paths(source_manifest_path: Optional[str]) -> Dict[str, str]:
69     """Load ``video_id -> local_path`` from the collector golden-source manifest.
70
71     The collector records source/proxy ownership and full-file MD5s. Only entries marked
72     ``present_local`` with a concrete ``local_path`` can drive local GPU extraction.
73     """
74
75     if not source_manifest_path:
76         return {}
77     with open(source_manifest_path, encoding="utf-8") as f:
78         data = json.load(f)
79     entries = data.get("videos", data) if isinstance(data, dict) else data
80     if not isinstance(entries, list):
81         raise ValueError(f"source manifest must contain a videos list:
{source_manifest_path}")
82
83     paths: Dict[str, str] = {}
84     for entry in entries:
85         if not isinstance(entry, dict):
86             continue
87         video_id = entry.get("video_id") or entry.get("name")
88         local_path = entry.get("local_path")
89         if not video_id or not local_path:
90             continue
91         if entry.get("source_status", "present_local") != "present_local":
92             continue
93         paths[str(video_id)] = str(local_path)
94     return paths
95
96
97 def attach_source_video_paths(
98     specs: Sequence[Dict[str, Any]], source_manifest_path: Optional[str]
99 ) -> int:
100     """Attach explicit ``video_path`` fields to manifest specs from the source manifest.
101
102     Returns the number of specs updated. The collector source manifest is authoritative
103     for local source/proxy video location and overrides derived/stale manifest paths.
104     """
105
106     source_paths = load_source_video_paths(source_manifest_path)
107     updated = 0
108     for spec in specs:
109         name = str(spec.get("name", ""))
110         source_path = source_paths.get(name)
111         if not source_path:
112             continue
113         if spec.get("video_path") == source_path:
114             continue
115         spec["video_path"] = source_path
116         updated += 1
117     return updated
118
119
120 def _filter_specs(
121     specs: Sequence[Dict[str, Any]], only: Sequence[str]
122 ) -> List[Dict[str, Any]]:
123     if not only:
124         return list(specs)
125     wanted = list(dict.fromkeys(only))
126     wanted_set = set(wanted)
127     filtered = [spec for spec in specs if str(spec.get("name")) in wanted_set]
128     found = {str(spec.get("name")) for spec in filtered}
129     missing = [name for name in wanted if name not in found]

```

```

130         if missing:
131             raise ValueError(f"--only requested unknown video(s): {'', '.join(missing)}")
132         return filtered
133
134
135     def _derive_video_path(spec: Dict[str, Any]) -> str:
136         """Original MP4 for a manifest entry ? explicit `video_path`, else derived from the
137         golden labels path (`X.rallies.csv` -> `X.MP4`, the golden-set naming
138         convention)."""
139         if spec.get("video_path"):
140             return str(spec["video_path"])
141         labels = str(spec["labels"])
142         if labels.endswith(".rallies.csv"):
143             return labels[: -len(".rallies.csv")] + ".MP4"
144         raise ValueError(
145             f"{spec.get('name')}: no video_path and labels {labels!r} isn't *.rallies.csv"
146         )
147
148     def extract_video(
149         spec: Dict[str, Any],
150         detector: Optional[Any] = None,
151         frame_skip: int = 3,
152         iou: float = 0.5,
153         log_every_sec: float = 15.0,
154         telemetry: Optional[Any] = None,
155         use_gpu: bool = True,
156         detector_model: Optional[str] = None,
157         max_gpu_temp_c: Optional[float] = None,
158     ) -> Dict[str, Any]:
159         """One golden video -> a feature record (shuttle + person features per candidate
160         window).
161
162         Returns `{name, fps, frame_width,
163         candidates:[{start,end,shuttle{ },person{ },label}], gts}``.
164         `detector` (anything with `detections_over_windows`) is injectable for GPU-free
165         tests;
166         when None a real `PersonDetector` is built (loads torchvision on first use).
167         `telemetry` (a `RunTelemetry` or None) records per-frame velocity + machine-health
168         snapshots
169         from the detection progress callback so a hard kill leaves a forensic runlog.
170         """
171         fps, fw = float(spec["fps"]), float(spec["frame_width"])
172         traj = str(spec["trajectory"])
173         points = TrackNetRunner.parse_trajectory_csv(traj)
174         intervals, x_shuttle = distill_local.build_candidates(traj, fps, fw)
175         gts = load_golden_gts(str(spec["labels"]))
176         labels = label_candidates(intervals, gts, iou)
177         # Net axis for the person two-sided split ? reuse rally_gate's camera-agnostic
178         estimate
179         # (parity with the shuttle net-crossing gate) rather than hard-coding an
180         orientation.
181         axis, net_px, _span = rally_gate.estimate_play_axis(points)
182         video_path = _derive_video_path(spec)
183         if not os.path.isfile(video_path):
184             raise FileNotFoundError(
185                 f"{spec.get('name')}: source video not found for fusion extraction:
186                 {video_path}"
187             )
188
189         if detector is None:
190             from backend.pipeline.detectors.person_detector import PersonDetector
191
192         indexing_cfg: Dict[str, Any] = {"use_gpu": use_gpu}
193         if detector_model:

```

```

187         indexing_cfg["detector_model"] = detector_model
188         detector = PersonDetector({"indexing": indexing_cfg})
189
190     # Single-pass person detection over ALL candidate windows (open the video ONCE, read
191     # forward ? no per-window seek). Then compute features per window (pure, no GPU).
192     n = len(intervals)
193     win_frames = [(round(s * fps), round(e * fps)) for (s, e) in intervals]
194     print(
195         f"[fusion_golden]    {spec['name']}: {n} windows ? person-detecting in one
pass...",
196         flush=True,
197     )
198     t0 = time.monotonic()
199     state = {"last": t0}
200
201     def _progress(done: int, total: int) -> None:
202         rec = None
203         if telemetry is not None:
204             rec = telemetry.snapshot(done, total, phase=spec["name"])
205         if max_gpu_temp_c is not None and rec is not None:
206             gpu = rec.get("gpu") or {}
207             temp = gpu.get("temp_c")
208             if temp is not None and float(temp) >= max_gpu_temp_c:
209                 raise RuntimeError(
210                     f"{spec['name']}: GPU temperature {float(temp):.1f}C reached "
211                     f"--max-gpu-temp-c {max_gpu_temp_c:.1f}; aborting before thermal
shutdown"
212                 )
213             now = time.monotonic()
214             if now - state["last"] >= log_every_sec or done >= total:
215                 el = now - t0
216                 rate = done / el if el > 0 else 0.0
217                 eta = (total - done) / rate if rate > 0 else 0.0
218                 pct = 100 * done // total if total else 0
219                 print(
220                     f"[fusion_golden]    {spec['name']}: detect frame {done}/{total} ({pct}%)
"
221                     f"| {el:.0f}s elapsed | ETA {eta:.0f}s",
222                     flush=True,
223                 )
224                 state["last"] = now
225
226     per_window = detector.detections_over_windows(
227         video_path, win_frames, frame_skip, progress_cb=_progress
228     )
229
230     candidates: List[Dict[str, Any]] = []
231     for i, ((s, e), xs) in enumerate(zip(intervals, x_shuttle)):
232         det = per_window[i]
233         det_fw = det.get("frame_width") or fw
234         det_fh = det.get("frame_height") or 0.0
235         # Normalise the (pixel) net coord to 0-1 on the play axis, matching the foot-
point
236
237         # normalisation fusion_features uses (x/width, y/height).
238         if axis == "x":
239             net_norm = net_px / det_fw if det_fw > 0 else 0.5
240         else:
241             net_norm = net_px / det_fh if det_fh > 0 else 0.5
242         sf, ef = win_frames[i]
243         shuttle_in = [p for p in points if sf <= p.frame <= ef]
244         person = ff.compute_person_features(
245             det["frames"],
246             shuttle_in,
247             det_fw,
248             det_fh,

```

```

248         court_polygon=None,
249         net=(axis, net_norm),
250     )
251     candidates.append(
252         {
253             "start": float(s),
254             "end": float(e),
255             "shuttle": {k: float(v) for k, v in zip(SHUTTLE_FEATURE_NAMES, xs)},
256             "person": {k: float(v) for k, v in person.items()},
257             "label": int(labels[i]),
258         }
259     )
260     return {
261         "name": spec["name"],
262         "fps": fps,
263         "frame_width": fw,
264         "candidates": candidates,
265         "gts": [[float(a), float(b)] for a, b in gts],
266     }
267
268
269     def run(
270         manifest_path: str,
271         out_dir: str,
272         frame_skip: int = 3,
273         iou: float = 0.5,
274         fresh: bool = False,
275         smallest_first: bool = False,
276         source_manifest: Optional[str] = None,
277         only: Sequence[str] = (),
278         missing_fusion_only: bool = False,
279         use_gpu: bool = True,
280         detector_model: Optional[str] = None,
281         max_gpu_temp_c: Optional[float] = None,
282     ) -> List[str]:
283         specs = load_resolved_manifest(manifest_path)
284         source_manifest = source_manifest or os.environ.get(SOURCE_MANIFEST_ENV)
285         if source_manifest:
286             updated = attach_source_video_paths(specs, source_manifest)
287             print(
288                 f"[fusion_golden] attached {updated}/{len(specs)} source video path(s) "
289                 f"from {source_manifest}",
290                 flush=True,
291             )
292         specs = _filter_specs(specs, only)
293         if smallest_first:
294             # Process small videos first for quick feedback (the slow full match lands
295             # last);
296             # the LOVO result is order-independent. Proxy size by the trajectory CSV (one
297             # row/frame).
298             specs = sorted(
299                 specs,
300                 key=lambda s: (
301                     os.path.getsize(s["trajectory"])
302                     if os.path.isfile(s.get("trajectory", ""))
303                     else 0
304                 ),
305             )
306         os.makedirs(out_dir, exist_ok=True)
307         # Run-telemetry black box: a long GPU run can be OS-killed with no traceback, so
308         # keep a tiny
309         # rotating runlog of velocity + machine health for the post-mortem. Constructing it
310         # must never
311         # break extraction, so guard the build and fall back to no telemetry on any error.
312         telem: Optional[RunTelemetry]

```

```

309     try:
310         telem = RunTelemetry(
311             os.path.join(out_dir, "run_telemetry.jsonl"), label="fusion_golden"
312         )
313     except Exception: # noqa: BLE001 - telemetry wiring must never break the run it
observes
314         telem = None
315     written: List[str] = []
316     for vi, spec in enumerate(specs, 1):
317         out = os.path.join(out_dir, f"{spec['name']}_golden_features.json")
318         existing = os.path.isfile(out)
319         existing_full = existing and is_full_fusion_feature_file(out)
320         # Resumable: skip a video whose full-fusion JSON already exists. A leaner
321         # windowing-only JSON is stale for A7, so re-extract it even without --fresh.
322         if missing_fusion_only and not fresh and existing_full:
323             print(
324                 f"[fusion_golden] ({vi}/{len(specs)}) {spec['name']}: full-fusion "
325                 f"features already present -> {out}",
326                 flush=True,
327             )
328             written.append(out)
329             continue
330         if not fresh and existing and existing_full:
331             print(
332                 f"[fusion_golden] ({vi}/{len(specs)}) {spec['name']}: already extracted
333                 "
334                 f"-> {out} (use --fresh to redo)",
335                 flush=True,
336             )
337             written.append(out)
338             continue
339         if not fresh and existing and not existing_full:
340             print(
341                 f"[fusion_golden] ({vi}/{len(specs)}) {spec['name']}: existing JSON "
342                 f"is not full-fusion schema; re-extracting...",
343                 flush=True,
344             )
345             print(
346                 f"[fusion_golden] ({vi}/{len(specs)}) {spec['name']}: extracting...",
347                 flush=True,
348             )
349         t0 = time.monotonic()
350         if telem is not None:
351             telem.event("video_start", spec["name"])
352         try:
353             data = extract_video(
354                 spec,
355                 frame_skip=frame_skip,
356                 iou=iou,
357                 telemetry=telem,
358                 use_gpu=use_gpu,
359                 detector_model=detector_model,
360                 max_gpu_temp_c=max_gpu_temp_c,
361             )
362         except Exception as e: # noqa: BLE001 - record machine state at death, then re-
raise
363             if telem is not None:
364                 telem.event("error", spec["name"], err=str(e))
365             raise
366         n_pos = sum(c["label"] for c in data["candidates"])
367         with open(out, "w", encoding="utf-8") as f:
368             json.dump(data, f, indent=2)
369         if telem is not None:
370             telem.event(
371                 "video_done",

```

```

371         spec["name"],
372         windows=len(data["candidates"]),
373         positives=n_pos,
374     )
375     print(
376         f"[fusion_golden] ({vi}/{len(specs)}) {spec['name']}: DONE in
{time.monotonic() - t0:.0f}s "
377         f"| {len(data['candidates'])} windows ({n_pos} positive) |
{len(data['gts'])} golden rallies "
378         f"-> {out}",
379         flush=True,
380     )
381     written.append(out)
382     return written
383
384
385 def main() -> None:
386     ap = argparse.ArgumentParser(
387         description="Extract golden-set shuttle+person features (fusion P3 GPU step)."
388     )
389     ap.add_argument("--manifest", default="output/human_lovo_manifest.json")
390     ap.add_argument("--out-dir", default="output")
391     ap.add_argument("--frame-skip", type=int, default=3)
392     ap.add_argument("--iou", type=float, default=0.5)
393     ap.add_argument(
394         "--fresh", action="store_true", help="re-extract even if a JSON already exists"
395     )
396     ap.add_argument(
397         "--smallest-first",
398         action="store_true",
399         help="process smaller videos first for quick feedback (LOVO is order-
independent)",
400     )
401     ap.add_argument(
402         "--source-manifest",
403         default=None,
404         help=(
405             "collector golden_source_videos.json; overrides video paths for source/proxy
"
406             f"videos (or set {SOURCE_MANIFEST_ENV})"
407         ),
408     )
409     ap.add_argument(
410         "--only",
411         action="append",
412         default=[],
413         metavar="VIDEO_ID",
414         help="process one manifest video by name; repeatable",
415     )
416     ap.add_argument(
417         "--missing-fusion-only",
418         action="store_true",
419         help="skip videos whose existing JSON already has full shuttle+person fusion
schema",
420     )
421     ap.add_argument(
422         "--cpu",
423         action="store_true",
424         help="run torchvision person detection on CPU instead of CUDA",
425     )
426     ap.add_argument(
427         "--detector-model",
428         default=None,
429         choices=[
430             "fasterrcnn_resnet50_fpn",

```



```

431         "fasterrcnn_mobilenet_v3_large_fpn",
432         "retinanet_resnet50_fpn",
433     ],
434     help="torchvision detector model; default keeps the configured Faster R-CNN
ResNet",
435 )
436 ap.add_argument(
437     "--max-gpu-temp-c",
438     type=float,
439     default=None,
440     help="abort extraction if telemetry sees GPU temperature at or above this
value",
441 )
442 args = ap.parse_args()
443 run(
444     args.manifest,
445     args.out_dir,
446     args.frame_skip,
447     args.iou,
448     fresh=args.fresh,
449     smallest_first=args.smallest_first,
450     source_manifest=args.source_manifest,
451     only=args.only,
452     missing_fusion_only=args.missing_fusion_only,
453     use_gpu=not args.cpu,
454     detector_model=args.detector_model,
455     max_gpu_temp_c=args.max_gpu_temp_c,
456 )
457
458
459 if __name__ == "__main__":
460     main()
461

```

4. assistant (2026-07-04T00:48:53.992Z)

Now I'll trace through the exact logic of the three conditional branches to determine the flag's impact.

****Logic Analysis:****

Lines 322-347 contain the core decision logic. Let me trace all 8 combinations of `(missing\_fusion\_only, existing, existing\_full)` with `fresh=False`:

missing_fusion_only	existing	existing_full	Line 322 match?	Line 330 match?	Line 338 match?	Behavior
T	T	T	YES	?	?	SKIP (line 328: continue)
T	T	F	NO	NO	YES	RE-EXTRACT (fall through)
T	F	F	NO	NO	NO	RE-EXTRACT (fall through)
T	F	T	NO	NO	NO	RE-EXTRACT (fall through)
F	T	T	NO	YES	?	SKIP (line 337: continue)
F	T	F	NO	NO	YES	RE-EXTRACT (fall through)
F	F	F	NO	NO	NO	RE-EXTRACT (fall through)
F	F	T	NO	NO	NO	RE-EXTRACT (fall through)

****Key observation:**** Line 322's condition is `if missing\_fusion\_only and not fresh and existing\_full:` ? it requires the flag to be True. When `missing\_fusion\_only=False`, this branch is never taken.

When `missing\_fusion\_only=True` and `existing=True, existing\_full=True` (row 1), line 322 matches and the video is SKIPPED with a log message about "full-fusion features already present".

When `missing\_fusion\_only=False` and `existing=True, existing\_full=True` (row 5), line 330

matches and the video is SKIPPED with a different log message about "already extracted".

****Critical insight:**** Both paths lead to the same outcome (SKIP), but with different log messages. The flag does NOT change extraction behavior ? it only changes the printed reason.

****VERDICT: CONFIRMED****

The `--missing-fusion-only` flag is a ****functional no-op****. It only changes the log message printed, not which videos are skipped or re-extracted. When `missing_fusion_only=True` and a video has full fusion features, line 322 prints "full-fusion features already present"; when False, line 330 prints "already extracted". Both branches call `continue` (skip extraction). The flag's help text claims it "skip[s] videos whose existing JSON already has full shuttle+person fusion schema," but that is ****already exactly what the default (no-flag) path does**** via line 330's `existing_full` check.

****Fix:**** Remove line 322?329 entirely and consolidate the two messages into one conditional, or make the flag actually change behavior (e.g., only skip if both `missing_fusion_only` AND `existing_full`, while the default only skips if not fresh).